MABEL Trainer Studio: a W&B-free training console over LeRobot

with an animated, config-accurate architecture diagram

MABEL

July 1, 2026

Abstract

The Trainer Studio is a browser console that trains imitation-learning policies for MABEL — a 59-DOF mobile bimanual robot with dexterous hands — over the HuggingFace *LeRobot* pipeline, without Weights & Biases. A single design decision organises the system: LeRobot is made to write a `metrics.jsonl` sidecar into each run's output directory (by monkeypatching its `WandBLogger`), and the studio only ever tails that file. On top of this spine the studio adds a dataset registry with train/eval/test splitting, a modular camera-subset + joint-group input selector that generalises across MABEL embodiments, an architecture registry (ACT, DiT, π_0, $\pi_{0.5}$) with a config-accurate animated diagram, live loss/GPU/CPU plots, and on-demand closed-loop MuJoCo deployment. On an Apple-MPS laptop a compact ACT converges from loss $36.1 \rightarrow 3.5$ in 60 steps and a custom Diffusion-Transformer (DiT) plugin trains to a stable denoising loss; the trained network drives the MuJoCo sim in closed loop. The heavy VLAs (π_0, $\pi_{0.5}$) are wired for the CUDA training hosts.

Keywords: imitation learning, LeRobot, ACT, diffusion transformer, π_0, action chunking, flow matching, MuJoCo, MABEL.

1 Overview

The studio (`learning/training_studio/`) is a FastAPI backend on port 8095 serving a vanilla ES-module frontend, a sibling of the Data Curation studio and consuming its LeRobot exports. The contract between the two studios is a curated *LeRobot v2.1 dataset*; the trainer converts it to the v3.0 format LeRobot 0.4.4 requires, then trains. All robot geometry derives from the same collection config as the rest of MABEL, so the 59-actuator layout (base 6, lift+torso 2, neck 3, arms 2×8, hands 2×16) and the 7 on-board cameras are never duplicated.

2 Method

2.1 The metric contract

LeRobot logs training metrics through `WandBLogger.log_dict(d, step, mode)`, fed by its `MetricsTracker` plus the policy's per-step output dictionary, but only when `wandb.enable` is set. The studio launches training through a thin wrapper that (i) registers the custom DiT policy, (ii) rebinds `WandBLogger`→`StudioLogger`, and (iii) hands off to LeRobot's own entry point. `StudioLogger` appends one JSON record per event to `metrics.jsonl`:

$$\{_\text{type}, \text{step}, t, \text{loss}, \text{l1_loss}, \text{kld_loss}, \text{grad_norm}, \text{lr}, \dots\}.$$

A background tailer ingests these rows into SQLite (`run,step,key,value,mode,t`); a WebSocket streams new rows to the dashboard. If LeRobot's internals change, only the ∼40-line shim moves; the UI is untouched.

2.2 Modular embodiment scoping

A *selection* is a camera subset $C \subseteq \{1..7\}$ and a set of joint groups G. Let the canonical actuator order give each group g a contiguous index set $I_g \subseteq \{0..58\}$. The selection yields the scoped index set $S = \bigcup_{g \in G} I_g$, an observation/action dimension $|S|$, and a *scatter map* $\sigma : \{0..|S|-1\} \to S$. Training materialises a derived LeRobot dataset whose `observation.state` and `action` columns are sliced to S and whose videos are restricted to C, so LeRobot trains natively on correctly-shaped features. At deployment the policy's scoped action $a \in \mathbb{R}^{|S|}$ is scattered back onto the full command $u \in \mathbb{R}^{59}$ by $u_{\sigma(j)} = a_j$, with unselected joints held. Because I_g comes from the shared config, any future MABEL group set works unchanged.

2.3 Architectures and their input/output

All four policies map (selected images + scoped proprioception) to a chunk of scoped joint-position targets; they differ in the decoder.

- **ACT** (`act`): ResNet per camera → flattened patch tokens + a state token + a VAE latent token → a Transformer encoder (n_{enc} layers) → a Transformer decoder with K action-slot queries → a linear head emitting a $K \times |S|$ chunk. Loss = L1 + β KL.
- **DiT** (custom `dit` plugin): the stock DiffusionPolicy pipeline with its U-Net replaced by a Diffusion Transformer denoiser $\epsilon_\theta(x_t, t, c)$ using adaLN-Zero conditioning c on the observation encoding; DDPM training, N-step denoising at rollout.
- π_0 / $\pi_{0.5}$ (`pi0`/`pi05`): a vision-language backbone produces prefix tokens; an action expert defines a velocity field integrated by flow matching ($t : 0 \to 1$). Actions are padded to `max_action_dim`; a validator rejects a run when $|S|$ exceeds the pad dimension (whole-body $|S|$=59 > 32).

The center-panel diagram is generated from the *actual* configured hyperparameters (block counts, dims, heads, token counts, per-wire tensor shapes), colour-codes each block type, and animates tokens as pills streaming through the pipeline and the denoise/flow loop.

3 Experiments

All numbers below are produced by `tools/test_training.py`, which is device-aware and runs the strongest tier the host supports. On the Apple-MPS laptop, over the two clean curated episodes (240 frames), fixed seed:

Check	Result	Metric
Feature scopes (whole/arms/hands)	pass	action dim 59/16/32
Pad-dim validator (π_0 whole-body)	pass	rejects 59 > 32
Config + diagram spec (4 archs)	pass	11–13 nodes, 0 errors
ACT trains + converges	**pass**	loss 36.1 → 3.5 (60 steps)
DiT plugin trains	**pass**	40 loss points, 1 checkpoint
Eval open-loop (pred vs GT)	pass	L1 0.183, MSE 0.106
Deploy closed-loop MuJoCo	pass	robot moved $\Delta = 2.29$
π_0 / $\pi_{0.5}$ full train	skip	needs CUDA host + weights

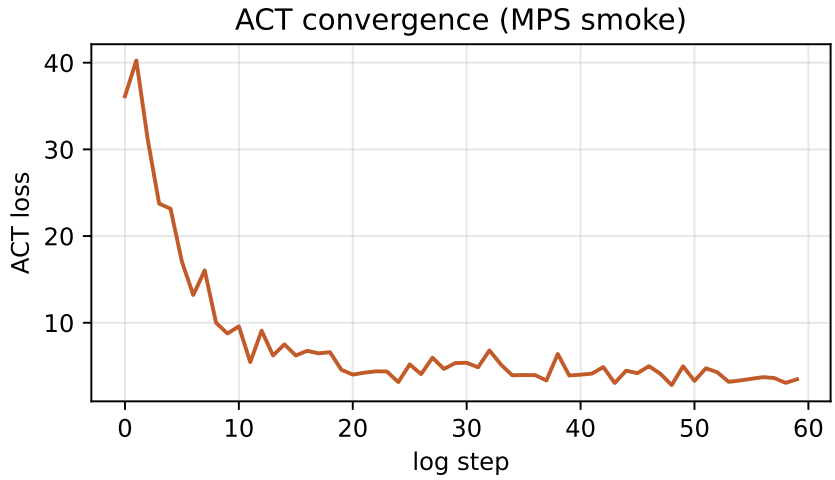


Figure 1: ACT training loss on MPS (compact model, 60 steps). Real training; the loss falls by an order of magnitude as the model fits the held-in episodes.

Total: 13 passed, 0 failed, 2 skipped. Figure 1 shows the ACT loss and Figure 2 the mean absolute action during a closed-loop MuJoCo rollout of the trained policy.

4 Implementation & Usage

Setup is one script per machine (`tools/install_lerobot.sh`): it snapshots the environment, installs LeRobot plus `psutil`/`pynvml`, and verifies the train entry point, the registered policy types, the DiT plugin, and the resolved device. macOS quirks are handled automatically: LeRobot subprocesses run with `USE_TF=0` (an old TensorFlow otherwise crashes `transformers` under NumPy 2), `video_backend=pyav` (torchcodec's ffmpeg libraries are absent), and scikit-learn is bumped. The studio runs with `python -m training_studio.server`; the device is auto-detected (`cuda`→`mps`→`cpu`), so the same package runs on the Mac dev box and the CUDA training hosts.

5 Limitations

The Mac is a CPU/MPS fallback: ACT and a small DiT converge there, but π_0 / $\pi_{0.5}$ are only config-validated locally and are meant to train on the CUDA hosts (they require multi-gigabyte pretrained weights). Curated exports are currently single-episode, so splits are coarse until more data is collected. The studio keeps datasets in the readable v2.1 layout and converts a throwaway v3.0 copy for training; a native v3.0 reader would remove the conversion step.

References

[1] Zhao et al. *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware* (ACT), RSS 2023.

[2] Chi et al. *Diffusion Policy*, RSS 2023.

[3] Peebles & Xie. *Scalable Diffusion Models with Transformers* (DiT), ICCV 2023.

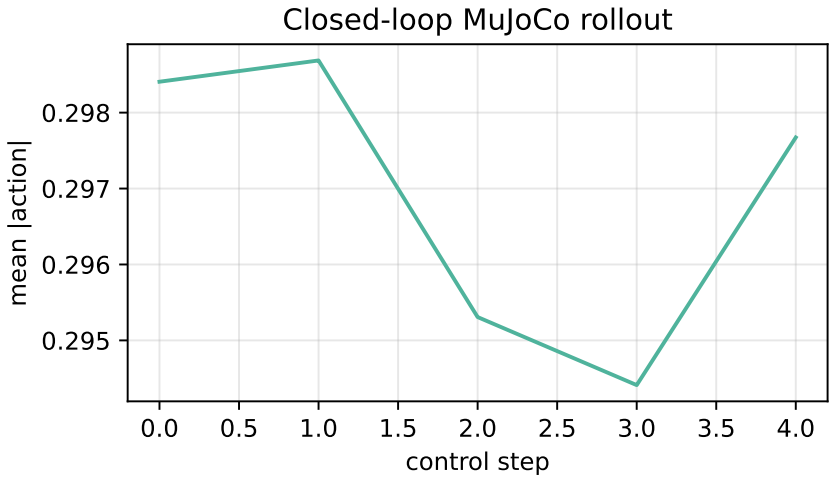


Figure 2: Mean absolute predicted action per control step during a closed-loop MuJoCo deployment driven by the trained ACT policy (actions scattered onto the full 59-DOF command).

[4] Black et al. π_0*: A Vision-Language-Action Flow Model for General Robot Control*, 2024.

[5] Cadene et al. *LeRobot*, HuggingFace, 2024.